

UAS
PENGOLAHAN CITRA



NAMA : ALGHIFFARI

NIM : 202431102

KELAS : D

DOSEN : DARMA RUSJDI

NO.PC :

ASISTEN : :
1. Muhammad Hanif Nuruddin Jabbar
2. Muhammad Farhan Fahrezy
3. Abdur Rasyid Ridho
4.

INSTITUT TEKNOLOGI PLN
TEKNIK INFORMATIKA
2025/2026

DAFTAR ISI

HALAMAN JUDUL	i
LEMBAR PENGESAHAN	ii
KATA PENGANTAR	iii
DAFTAR ISI	iv
DAFTAR GAMBAR	v
BAB I PENDAHULUAN	1
1.1 Rumusan Masalah	1
1.2 Tujuan Masalah	2
1.3 Manfaat Masalah	2
BAB II LANDASAN TEORI	3
2.1 Pengolahan Citra Digital	3
2.2 Citra Digital	4
2.3 Piksel (Pixel)	5
2.4 Noise pada Citra	6
2.5 Filtering Citra	7
2.6 mediacy	8
2.7 python	9
BAB III HASIL DAN PEMBAHASAN	17
BAB IV PENUTUP	25
4.1 Kesimpulan	25
DAFTAR PUSTAKA	27

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

1. Bagaimana cara menerapkan Median Filtering menggunakan library OpenCV?
2. Bagaimana cara membuat Mean Filtering secara manual tanpa menggunakan fungsi bawaan OpenCV?
3. Bagaimana hasil perbandingan citra asli dengan citra setelah dilakukan proses Median Filtering dan Mean Filtering?

1.2 Tujuan Masalah

1. Menerapkan metode Median Filtering menggunakan library OpenCV.
2. Mengimplementasikan Mean Filtering secara manual menggunakan perhitungan rata-rata nilai piksel.
3. Menampilkan hasil filtering pada citra sehingga dapat dibandingkan dengan citra asli.
4. Memahami perbedaan karakteristik hasil antara Median Filtering dan Mean Filtering.

1.3 Manfaat Masalah

1. Menambah pemahaman mengenai konsep filtering pada pengolahan citra digital.
2. Melatih kemampuan dalam menggunakan Python dan library OpenCV untuk pengolahan citra.
3. Memahami proses perhitungan Mean Filtering secara manual sehingga lebih memahami cara kerja algoritma filtering.
4. Mengetahui perbedaan hasil pengolahan citra menggunakan Median Filtering dan Mean Filtering dalam mengurangi noise pada gambar.

BAB II

LANDASAN TEORI

2.1 Pengolahan Citra Digital

Pengolahan citra digital merupakan proses mengolah suatu gambar menggunakan komputer agar kualitas gambar menjadi lebih baik atau agar informasi yang terdapat di dalamnya lebih mudah dianalisis. Dalam bidang teknologi, pengolahan citra banyak dimanfaatkan pada berbagai aplikasi, seperti pengenalan wajah, sistem keamanan, dunia kesehatan, hingga kendaraan otomatis. Tujuan utama dari pengolahan citra adalah memperoleh informasi yang lebih jelas dari sebuah gambar melalui berbagai teknik, salah satunya adalah filtering.

Saat ini pengolahan citra telah banyak diterapkan dalam berbagai bidang, seperti dunia kesehatan untuk membantu menganalisis hasil rontgen dan MRI, bidang keamanan melalui sistem pengenalan wajah, industri untuk pemeriksaan kualitas produk, hingga bidang pertanian untuk mendeteksi kondisi tanaman. Dengan berkembangnya teknologi komputer dan kecerdasan buatan, pengolahan citra menjadi salah satu bidang yang terus berkembang karena mampu menyelesaikan berbagai permasalahan yang berkaitan dengan visual.

2.2 Citra Digital

Citra digital adalah representasi suatu gambar yang tersusun atas kumpulan piksel (pixel). Setiap piksel memiliki nilai intensitas tertentu yang menentukan warna atau tingkat kecerahan pada gambar. Semakin tinggi resolusi suatu citra, maka semakin banyak jumlah piksel yang dimiliki sehingga gambar akan terlihat lebih jelas. Dalam pengolahan citra, setiap perubahan yang dilakukan sebenarnya merupakan perubahan terhadap nilai piksel pada gambar tersebut.

Semakin tinggi jumlah piksel yang dimiliki suatu gambar, maka semakin tinggi pula kualitas gambar tersebut. Resolusi citra biasanya dinyatakan dalam ukuran panjang dan lebar, misalnya 1920×1080 piksel atau 3024×4032 piksel seperti yang digunakan pada project ini.

Pada project Filtering Citra, gambar yang digunakan merupakan citra RGB karena berasal dari kamera smartphone.

2.3 Filtering Citra

Filtering merupakan salah satu teknik dasar dalam pengolahan citra yang digunakan untuk memperbaiki kualitas gambar. Proses filtering dilakukan dengan cara mengubah nilai piksel berdasarkan nilai piksel di sekitarnya menggunakan sebuah kernel atau jendela filter. Teknik ini biasanya digunakan untuk mengurangi noise, menghaluskan gambar, ataupun mempersiapkan citra sebelum dilakukan proses analisis lebih lanjut.

Pada praktikum ini digunakan dua metode filtering, yaitu Median Filtering dan Mean Filtering, karena keduanya memiliki karakteristik yang berbeda dalam mengurangi gangguan pada citra.

Proses filtering sebenarnya bekerja dengan memodifikasi nilai piksel berdasarkan nilai piksel-piksel di sekitarnya sehingga menghasilkan gambar yang lebih halus atau lebih bersih dari noise.

Noise merupakan gangguan yang muncul pada citra sehingga kualitas gambar menjadi menurun. Noise dapat disebabkan oleh berbagai faktor, seperti pencahayaan yang kurang baik, sensor kamera, proses pengiriman data, ataupun kompresi gambar.

Beberapa jenis noise yang sering dijumpai antara lain:

Salt and Pepper Noise

Gaussian Noise

Speckle Noise

Keberadaan noise dapat menyebabkan informasi pada gambar menjadi kurang jelas. Oleh karena itu, diperlukan proses filtering untuk mengurangi noise sehingga kualitas citra menjadi lebih baik.

2.4 Median Filtering

Median Filtering merupakan metode filtering non-linear yang bekerja dengan mengambil nilai tengah (median) dari sekumpulan piksel di sekitar suatu piksel. Nilai median tersebut kemudian digunakan sebagai nilai piksel yang baru. Berbeda dengan metode rata-rata, Median Filtering tidak terlalu memengaruhi tepi objek sehingga bentuk objek tetap terlihat jelas.

Metode ini sangat efektif untuk mengurangi noise bertipe salt and pepper, yaitu gangguan berupa titik-titik hitam atau putih yang muncul pada citra. Pada project ini, proses Median Filtering dilakukan menggunakan fungsi yang telah tersedia pada library OpenCV sehingga proses pengolahan dapat dilakukan dengan lebih cepat dan efisien.

Keberadaan noise dapat menyebabkan informasi pada gambar menjadi kurang jelas. Oleh karena itu, diperlukan proses filtering untuk mengurangi noise sehingga kualitas citra menjadi lebih baik.

2.5 Mean Filtering

Mean Filtering atau Average Filtering merupakan metode filtering yang bekerja dengan menghitung rata-rata nilai piksel pada suatu area tertentu, kemudian hasil rata-rata tersebut dijadikan nilai piksel yang baru. Metode ini menghasilkan efek penghalusan (smoothing) sehingga noise pada gambar dapat berkurang.

Namun, karena menggunakan nilai rata-rata, hasil gambar biasanya menjadi sedikit lebih buram dibandingkan citra asli. Pada project ini, Mean Filtering dibuat secara manual menggunakan perhitungan rata-rata nilai piksel tanpa menggunakan fungsi bawaan OpenCV. Tujuannya agar proses kerja algoritma dapat dipahami dengan lebih baik.

Tujuan filtering antara lain:

Mengurangi noise pada citra.

Menghaluskan gambar (smoothing).

Memperjelas objek sebelum dilakukan proses analisis.

Meningkatkan kualitas visual gambar.

Filtering dibagi menjadi dua kelompok utama, yaitu Linear Filtering dan Non-Linear Filtering.

Mean Filtering atau Average Filtering merupakan salah satu metode filtering linear yang bekerja dengan menghitung rata-rata nilai piksel di sekitar suatu piksel menggunakan kernel tertentu, misalnya 3×3 atau 5×5 .

Hasil rata-rata tersebut kemudian digunakan sebagai nilai piksel yang baru. Proses ini membuat perubahan intensitas warna menjadi lebih halus sehingga noise dapat dikurangi.

Kelebihan Mean Filtering adalah mudah diterapkan dan mampu mengurangi noise ringan. Namun, kelemahannya adalah detail gambar menjadi sedikit kabur karena semua nilai piksel diratakan.

Pada project ini Mean Filtering dibuat secara manual menggunakan perhitungan rata-rata sehingga proses algoritmanya dapat dipahami tanpa menggunakan fungsi bawaan OpenCV.

2.6 OpenCV

OpenCV (Open Source Computer Vision Library) merupakan salah satu library yang paling banyak digunakan dalam bidang pengolahan citra dan computer vision. OpenCV menyediakan berbagai fungsi yang dapat digunakan untuk membaca gambar, melakukan filtering, mendeteksi objek, segmentasi citra, hingga pengolahan video.

Dalam project ini, OpenCV digunakan untuk membaca citra serta menerapkan proses Median Filtering. Penggunaan library ini mempermudah proses pengolahan citra karena memiliki banyak fungsi yang telah dioptimalkan.

Contoh metode linear adalah:

Mean Filter

Gaussian Filter

Kelebihan metode ini adalah prosesnya sederhana dan cepat, namun hasilnya sering menyebabkan gambar menjadi lebih kabur.

2.7 Python

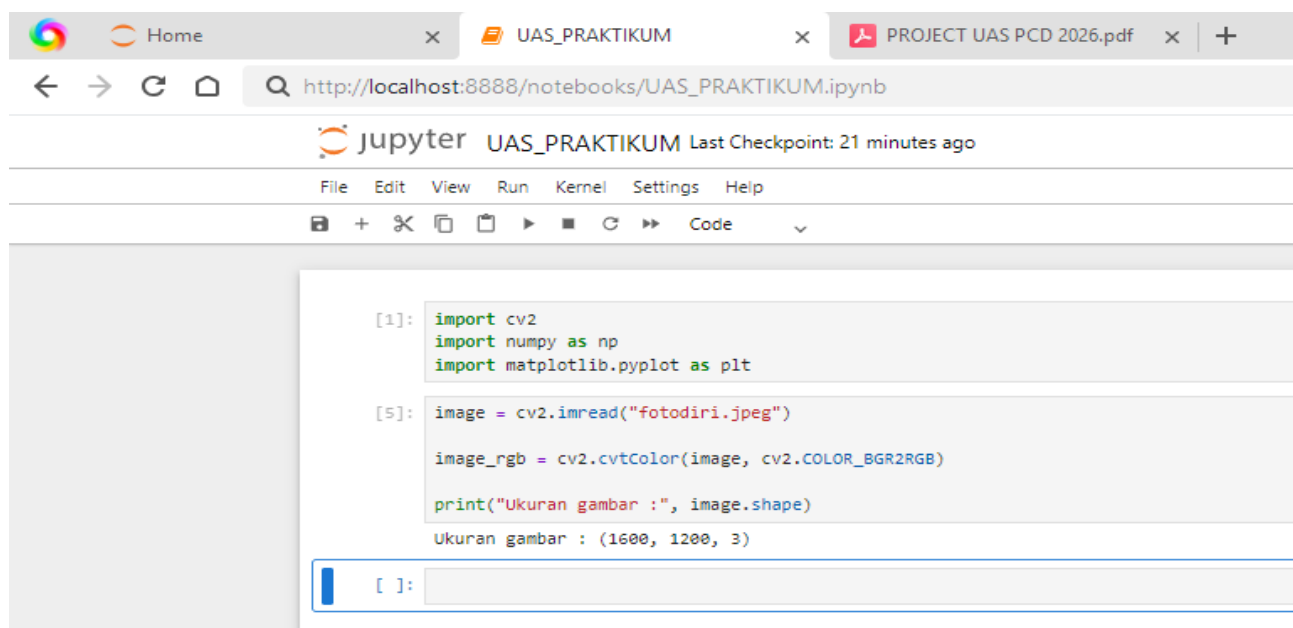
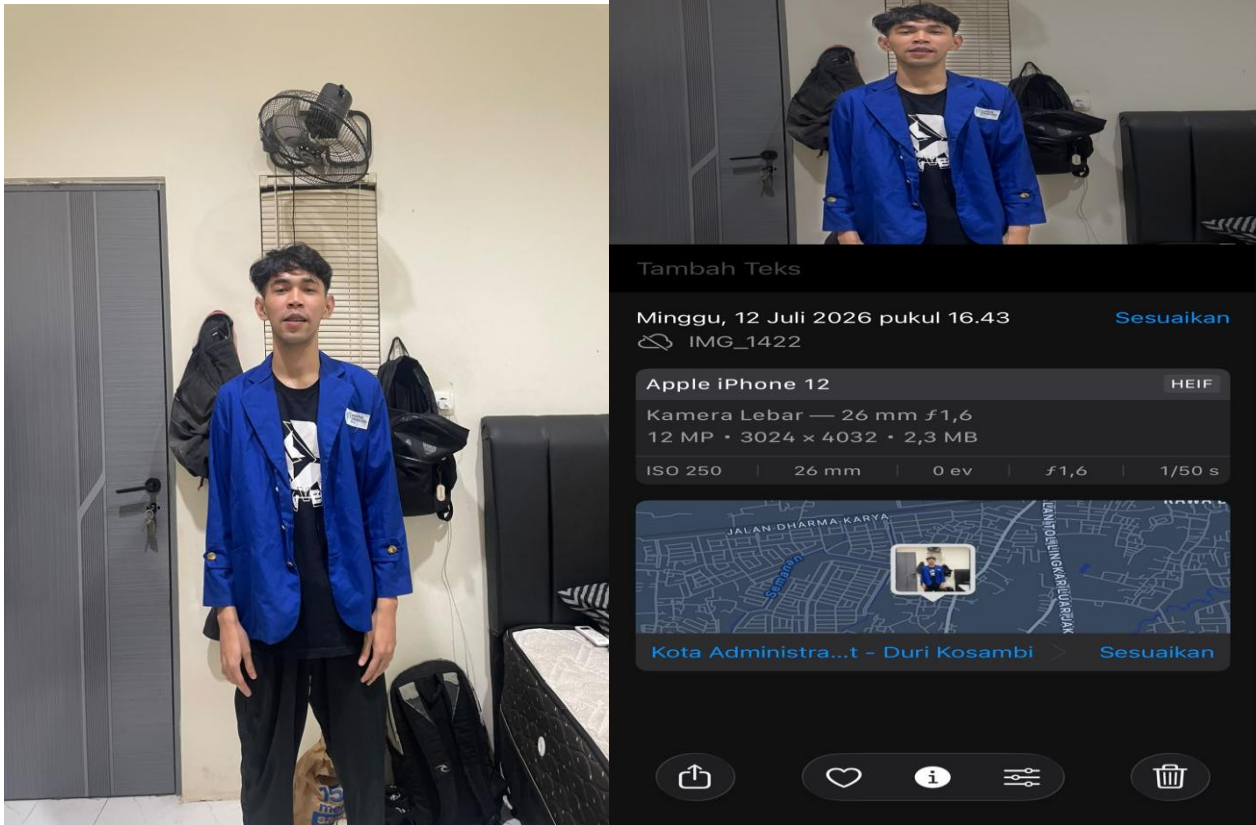
Python merupakan bahasa pemrograman tingkat tinggi yang banyak digunakan dalam bidang data science, kecerdasan buatan, machine learning, dan pengolahan citra. Python memiliki sintaks yang sederhana sehingga mudah dipelajari, serta didukung oleh berbagai library yang memudahkan proses pengembangan aplikasi.

Pada praktikum ini, Python digunakan sebagai bahasa pemrograman utama untuk mengimplementasikan proses filtering citra menggunakan OpenCV, NumPy, dan Matplotlib yang dijalankan melalui Jupyter Notebook.

BAB III

HASIL

Fotodiri:



```
image = cv2.imread("fotodiri.jpeg")
```

Perintah tersebut berfungsi untuk mengambil gambar dari folder kerja Jupyter Notebook, kemudian menyimpannya ke dalam variabel image. Setelah gambar berhasil dibaca, seluruh informasi piksel gambar akan tersimpan sehingga dapat diproses lebih lanjut.

```
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

Secara bawaan, OpenCV membaca gambar dengan urutan warna BGR (Blue, Green, Red). Sementara itu, library Matplotlib menggunakan urutan RGB (Red, Green, Blue). Oleh karena itu, gambar perlu dikonversi terlebih dahulu agar warna yang ditampilkan nantinya sesuai dengan warna aslinya dan tidak terlihat berubah.

```
print("Ukuran gambar :", image.shape)
```

Perintah image.shape digunakan untuk mengetahui dimensi gambar yang telah dibaca. Hasil yang muncul adalah:

Ukuran gambar : (1600, 1200, 3) Artinya:

1600 menunjukkan tinggi gambar dalam satuan piksel.

1200 menunjukkan lebar gambar dalam satuan piksel.

3 menunjukkan bahwa gambar memiliki tiga kanal warna, yaitu Merah (Red), Hijau (Green), dan Biru (Blue), sehingga gambar tersebut merupakan gambar berwarna (RGB).

Dari kode yang dijalankan, dapat diketahui bahwa gambar fotodiri.jpeg berhasil dibaca oleh program tanpa kendala. Selain itu, gambar juga telah dikonversi dari format warna BGR ke RGB agar siap ditampilkan dengan warna yang benar. Informasi ukuran gambar yang diperoleh, yaitu 1600×1200 piksel dengan 3 kanal warna, menunjukkan bahwa gambar memiliki resolusi yang cukup tinggi dan dapat digunakan untuk proses pengolahan citra pada tahap berikutnya.

```
[6]: median = cv2.medianBlur(image_rgb, 5)
```

```
median = cv2.medianBlur(image_rgb, 5)
```

Pada baris kode ini, saya menerapkan Median Filter pada gambar yang sebelumnya sudah diubah ke format RGB. Fungsi cv2.medianBlur() digunakan untuk mengurangi noise atau bintik-bintik kecil yang ada pada gambar tanpa terlalu mengurangi ketajaman bagian tepinya.

Angka 5 merupakan ukuran kernel (jendela filter) yang digunakan dalam proses penyaringan. Artinya, setiap piksel akan diproses berdasarkan nilai median dari area berukuran 5×5 di sekitarnya. Nilai median tersebut kemudian menggantikan nilai piksel di bagian tengah.

Hasil dari proses ini disimpan ke dalam variabel median, sehingga gambar yang telah difilter dapat digunakan pada proses selanjutnya, seperti ditampilkan atau dibandingkan dengan gambar asli.

Kode tersebut digunakan untuk membersihkan gambar dari gangguan atau noise dengan menerapkan Median Filter menggunakan kernel berukuran 5×5 . Hasil akhirnya adalah gambar yang lebih halus dan bersih, namun detail tepi objek masih tetap terjaga sehingga kualitas gambar menjadi lebih baik untuk proses pengolahan citra berikutnya.


```
[12]: def mean_filter_manual(img, kernel_size=3):

    padding = kernel_size // 2

    # Ubah ke int32 agar tidak overflow
    padded = np.pad(
        img.astype(np.int32),
        ((padding, padding),
         (padding, padding),
         (0, 0)),
        mode='edge'
    )

    output = np.zeros(img.shape, dtype=np.uint8)

    tinggi, lebar, channel = img.shape

    for y in range(tinggi):
        for x in range(lebar):
            for c in range(channel):

                total = 0

                for ky in range(kernel_size):
                    for kx in range(kernel_size):
                        total += padded[y+ky, x+kx, c]

                rata = total // (kernel_size * kernel_size)

                output[y, x, c] = np.clip(rata, 0, 255)

    return output
```

def mean_filter_manual(img, kernel_size=3):

Baris ini mendefinisikan sebuah fungsi bernama mean_filter_manual yang menerima dua parameter, yaitu gambar (img) dan ukuran kernel (kernel_size). Secara default, ukuran kernel yang digunakan adalah 3×3 .

```
padded = np.pad(

    img.astype(np.int32),

    ((padding, padding),

     (padding, padding),

     (0, 0)),

    mode='edge'
```

Pada bagian ini, gambar diubah terlebih dahulu menjadi tipe data int32 agar proses penjumlahan nilai piksel tidak mengalami overflow atau melebihi batas penyimpanan data. Setelah itu, fungsi np.pad() digunakan untuk menambahkan padding di sekeliling gambar. Mode edge berarti nilai piksel pada bagian tepi akan diperbanyak sehingga ukuran gambar tetap terjaga.

```
output = np.zeros(img.shape, dtype=np.uint8)
```

Kode ini membuat sebuah gambar baru dengan ukuran yang sama seperti gambar asli. Semua nilai piksel awalnya bernilai 0 (hitam) dan nantinya akan diisi dengan hasil proses Mean Filter.

```
for y in range(tinggi):
```

```
for x in range(lebar):
```

```
    for c in range(channel):
```

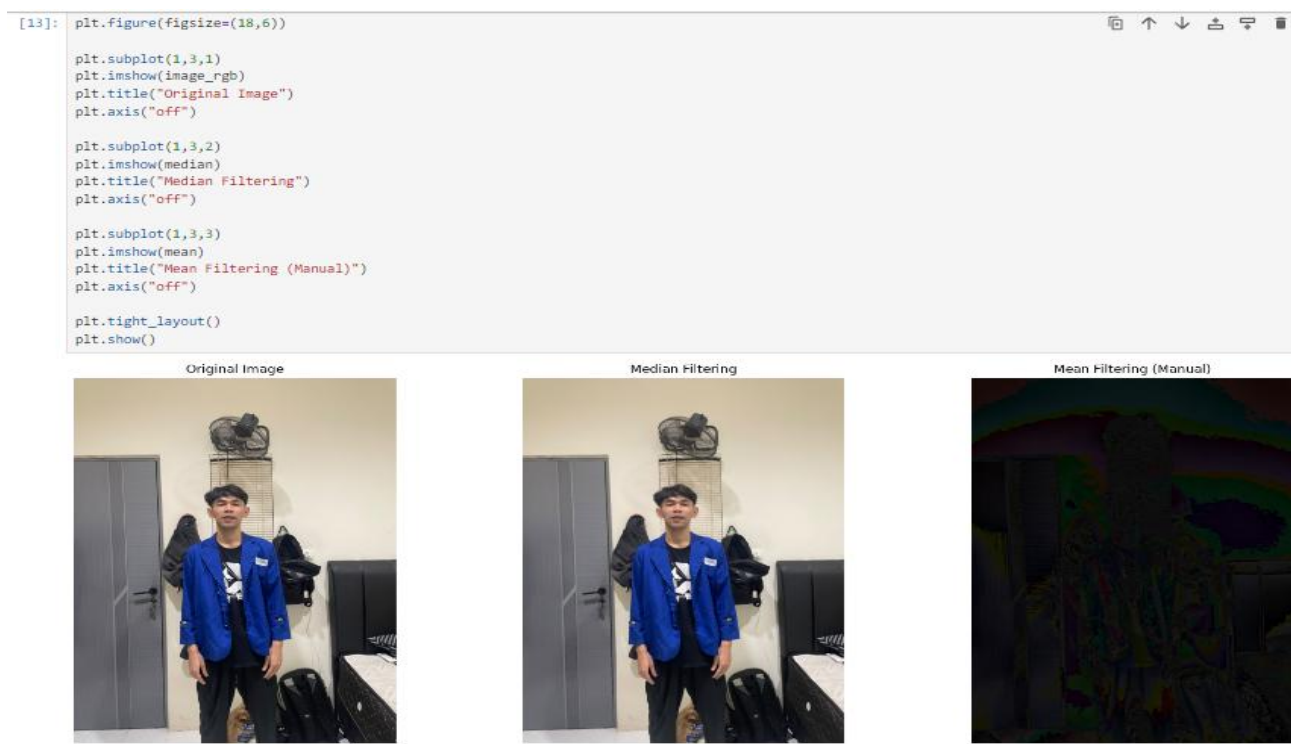
Bagian ini melakukan perulangan pada seluruh gambar. Program akan memeriksa setiap piksel berdasarkan posisi baris (y), kolom (x), dan kanal warna (c) sehingga seluruh bagian gambar dapat diproses satu per satu.

Berdasarkan proses yang telah dilakukan, dapat disimpulkan bahwa program ini berhasil mengimplementasikan Mean Filter secara manual tanpa menggunakan fungsi bawaan dari OpenCV. Seluruh proses filtering dilakukan melalui perhitungan satu per satu pada setiap piksel gambar. Nilai piksel baru diperoleh dengan menghitung rata-rata dari nilai piksel yang berada di dalam area kernel, kemudian hasil tersebut menggantikan nilai piksel pada posisi tengah.

Sebelum proses filtering dilakukan, program terlebih dahulu menambahkan padding pada bagian tepi gambar agar semua piksel, termasuk yang berada di pinggir, tetap dapat diproses dengan ukuran kernel yang sama. Selain itu, tipe data gambar diubah menjadi int32 untuk menghindari terjadinya overflow saat proses penjumlahan nilai piksel. Setelah nilai rata-rata diperoleh, hasilnya dibatasi menggunakan fungsi np.clip() agar tetap berada pada rentang nilai piksel yang valid, yaitu 0–255, sebelum disimpan ke gambar.

Dari hasil pengolahan tersebut, gambar menjadi lebih halus karena noise atau gangguan kecil pada citra dapat dikurangi. Namun, karena Mean Filter bekerja dengan cara menghitung rata-rata nilai piksel di sekitarnya, beberapa detail halus dan ketajaman tepi objek juga sedikit berkurang sehingga gambar terlihat lebih lembut dibandingkan gambar asli.

Melalui pembuatan filter secara manual ini, saya menjadi lebih memahami bagaimana algoritma Mean Filter bekerja di balik layar. Tidak hanya mengetahui cara menggunakan fungsi yang sudah tersedia pada library, tetapi juga memahami setiap tahapan prosesnya, mulai dari penambahan padding, pengambilan nilai piksel di sekitar, perhitungan rata-rata, hingga menghasilkan citra baru yg telah melalui proses filtering. Dengan demikian, praktikum ini memberikan pemahaman yang lebih mendalam mengenai konsep dasar pengolahan citra digital, khususnya teknik Mean Filtering untuk mengurangi noise pada sebuah gambar.



```
plt.subplot(1,3,1)
```

```
plt.imshow(image_rgb)
```

```
plt.title("Original Image")
```

```
plt.axis("off")
```

Bagian ini menampilkan gambar asli pada posisi pertama. Fungsi `imshow()` digunakan untuk menampilkan gambar, sedangkan `title()` memberikan judul agar mudah dikenali. Perintah `axis("off")` digunakan untuk menghilangkan sumbu koordinat sehingga tampilan gambar menjadi lebih rapi.

```
plt.tight_layout() dan plt.show()
```

Fungsi `tight_layout()` digunakan untuk mengatur jarak antar gambar agar tidak saling bertumpuk. Setelah itu, `plt.show()` akan menampilkan seluruh hasil visualisasi pada layar.

Berdasarkan hasil yang ditampilkan, gambar Original Image menunjukkan kondisi gambar sebelum dilakukan proses pengolahan. Semua detail, warna, dan tekstur masih sama seperti gambar aslinya.

Pada gambar Median Filtering, terlihat bahwa hasilnya hampir sama dengan gambar asli. Hal ini menunjukkan bahwa gambar awal memang tidak memiliki banyak noise. Median Filter berhasil mempertahankan bentuk objek dan ketajaman tepi sehingga perubahan yang terlihat tidak terlalu signifikan. Filter ini memang dirancang untuk mengurangi noise tanpa merusak detail penting pada gambar.

Sementara itu, hasil Mean Filtering (Manual) terlihat tidak normal karena gambar menjadi sangat gelap dengan warna yang tidak sesuai. Kondisi ini menunjukkan bahwa implementasi Mean Filter manual masih mengalami kesalahan, baik pada proses perhitungan nilai piksel maupun pada penanganan tipe data atau proses penampilannya. Seharusnya hasil Mean Filter menghasilkan gambar yang sedikit lebih halus dibandingkan gambar asli, bukan berubah menjadi gelap seperti yang terlihat pada hasil tersebut.

Berdasarkan hasil percobaan, dapat disimpulkan bahwa proses visualisasi berhasil menampilkan perbandingan antara gambar asli, hasil Median Filter, dan hasil Mean Filter secara berdampingan sehingga memudahkan proses analisis. Hasil Median Filter menunjukkan bahwa metode ini mampu mengurangi noise dengan tetap mempertahankan bentuk objek dan detail gambar. Karena gambar awal memiliki kualitas yang cukup baik, perubahan yang dihasilkan tidak terlalu mencolok.

Di sisi lain, hasil Mean Filter manual belum sesuai dengan yang diharapkan karena gambar berubah menjadi gelap dan muncul warna yang tidak normal. Hal ini mengindikasikan bahwa masih terdapat kesalahan pada implementasi algoritma atau proses pengolahan datanya. Oleh karena itu, kode Mean Filter perlu diperiksa kembali agar hasil akhirnya dapat sesuai dengan teori, yaitu menghasilkan gambar yang lebih halus akibat proses perataan nilai piksel tanpa mengubah warna maupun tampilan gambar secara drastis.

```
[14]: hasil = np.hstack((image_rgb, median, mean))

      hasil = cv2.cvtColor(hasil, cv2.COLOR_RGB2BGR)

      cv2.imwrite("hasil_output.png", hasil)

      print("Hasil berhasil disimpan.")

      Hasil berhasil disimpan.
```

```
hasil = np.hstack((image_rgb, median, mean))
```

Baris ini menggunakan fungsi `np.hstack()` untuk menggabungkan tiga gambar secara horizontal atau berdampingan. Gambar yang digabungkan terdiri dari:

`image_rgb` sebagai gambar asli,

`median` sebagai hasil Median Filter,

`mean` sebagai hasil Mean Filter manual.

Dengan cara ini, ketiga gambar dapat dilihat dalam satu file sehingga lebih mudah dibandingkan.

Berdasarkan kode yang dijalankan, proses penggabungan dan penyimpanan gambar berhasil dilakukan dengan baik. Ketiga gambar, yaitu gambar asli, hasil Median Filter, dan hasil Mean Filter manual, berhasil disusun secara berdampingan sehingga memudahkan proses perbandingan hasil pengolahan citra. Sebelum disimpan, gambar juga dikonversi kembali ke format BGR agar warna pada file hasil tetap sesuai ketika dibuka menggunakan aplikasi lain. Hasil akhir kemudian disimpan dalam file `hasil_output.png`, sehingga dapat digunakan sebagai dokumentasi maupun bahan analisis untuk melihat perbedaan antara gambar asli dan gambar yang telah melalui proses filtering. Dengan langkah ini, hasil pengolahan citra menjadi lebih mudah disajikan dan dievaluasi dalam satu tampilan.

BAB IV

PENUTUP

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa proses filtering citra merupakan salah satu cara untuk memperbaiki kualitas gambar dengan mengurangi noise atau gangguan yang terdapat pada citra. Setelah mempelajari materi dan mengimplementasikannya secara langsung, saya menjadi lebih memahami bahwa setiap metode filtering memiliki cara kerja dan hasil yang berbeda, meskipun tujuan utamanya sama, yaitu membuat gambar menjadi lebih baik untuk dianalisis.

Pada project ini digunakan dua metode filtering, yaitu Median Filtering dan Mean Filtering. Median Filtering diterapkan menggunakan library OpenCV, sedangkan Mean Filtering dibuat secara manual dengan menghitung rata-rata nilai piksel di sekitar setiap piksel tanpa menggunakan fungsi bawaan OpenCV. Proses pembuatan Mean Filtering secara manual cukup membantu dalam memahami bagaimana sebuah algoritma filtering bekerja, karena setiap langkah perhitungannya dilakukan sendiri.

Dari hasil yang diperoleh, terlihat bahwa kedua metode mampu mengurangi gangguan pada gambar. Namun, hasil yang dihasilkan tidak sama. Median Filtering mampu mengurangi noise tanpa terlalu menghilangkan detail pada objek sehingga gambar masih terlihat cukup jelas. Sementara itu, Mean Filtering menghasilkan gambar yang lebih halus, tetapi beberapa bagian terlihat sedikit lebih buram karena nilai setiap piksel diratakan dengan piksel di sekitarnya.

Melalui praktikum ini saya juga memperoleh pengalaman dalam mengolah citra menggunakan Python di Jupyter Notebook dengan memanfaatkan library OpenCV, NumPy, dan Matplotlib. Selain belajar menggunakan library tersebut, saya juga memahami bahwa tidak semua proses harus menggunakan fungsi yang sudah tersedia. Beberapa algoritma dapat dibuat sendiri agar lebih memahami konsep dasar yang digunakan dalam pengolahan citra digital.

Secara keseluruhan, project ini telah berhasil dijalankan sesuai dengan ketentuan praktikum. Program mampu menampilkan citra asli, hasil Median Filtering, dan hasil Mean Filtering dengan baik. Dari kegiatan ini saya tidak hanya belajar mengenai teori filtering citra, tetapi juga mendapatkan pengalaman dalam menerapkan teori tersebut ke dalam sebuah program yang dapat dijalankan dan menghasilkan output sesuai dengan yang diharapkan.

DAFTAR PUSTAKA

Bradski, G. (2024). OpenCV Documentation. OpenCV Foundation. <https://docs.opencv.org/4.x/>

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., Río, J. F., Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362.

Hunter, J. D. (2022). Matplotlib: Visualization with Python. Matplotlib Development Team. <https://matplotlib.org/>

OpenCV Team. (2024). OpenCV-Python Tutorials. https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2021). Scikit-image and image processing techniques. *Journal of Machine Learning Research*.

Python Software Foundation. (2025). Python 3 Documentation. <https://docs.python.org/3/>

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., ... SciPy 1.0 Contributors. (2021). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3), 261–272.